

FRED TALKS

11th May 2026

CONTENTS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

The lies I used to tell myself

CATE HALL · 1900 WORDS

Reimagining LinkedIn's search tech stack

FEDOR BORISYUK · 3443 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system.**

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

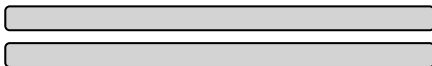
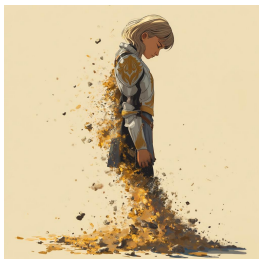
↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

↩

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · [SOURCE](#)



1. What doesn't kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We'd had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can't say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn't want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There's a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great [“50 Things” style post](#) a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: “Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless.”

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I “just knew” with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find \$20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing “well.” Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done “well,” but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really “do everything the way they do anything” — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter ... and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it “1 in 1000” matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it “well,” I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're

doing affects your ability to do it year in and year out, and dismissing this as “selfish” doesn’t make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. “All people are insane. They will do anything at any time, and God help anybody who looks for reasons.”

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they’d do next, seemed like a fool’s errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn’t fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People’s behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what’s rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can’t buy happiness.

The good version of this advice is: You’ll still have to work at being happy, even if you’re rich. Money in your bank account won’t do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there's research showing money doesn't buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, “I just do what feels right.” And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why “wisdom is knowledge that can't be transmitted.” Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, *You Can Just Do Things*, is available for purchase.

Reimagining LinkedIn's search tech stack

FEDOR BORISYUK · 31 JAN 2026 · [SOURCE](#)

Co-authors: [Jiahao Xu](#), [Xiaojing Ma](#), [Sriram Vasudevan](#), [Muchen Wu](#), [Rachel Zheng](#), [Benjamin Le](#), [Shaobo Zhang](#), [Sarang Metkar](#), [Rupesh Gupta](#), [Qianqi Kay Shen](#), [Ali Hooshmand](#), [David Nicolás Racca](#), [Vivek Katarya](#), [Kayhan Behdin](#), [Igor Lapchuk](#), [Xueying Lu](#), [Lingyu\(Claire\) Zhang](#), [Gokulraj Mohanasundaram](#), [Juan Pablo Bottaro](#), [Lily\(Jiayu\) Li](#), [Yanbo Li](#), [Guoyao L.](#), [Caleb Johnson](#), and [Sundara Raman Ramachandran](#)

At LinkedIn, our mission is to connect professionals to opportunity. Search plays a central role in this by helping members discover jobs, people, and knowledge that move their careers forward. As the professional landscape evolves, we strive to deliver search experiences that feel relevant, intuitive, personalized, and predictive of what truly matters, from applying to the right job to forming the right connection. That’s why we’ve recently introduced [AI Job Search](#) and [AI-powered People Search](#), which go beyond keyword matching and better understand member intent.

These products reimagine LinkedIn search, using large language models (LLMs) to create a semantic search experience. Instead of relying on exact word overlap between queries and postings, it interprets natural language to infer user goals and preferences. This semantic representation allows for more flexible and accurate retrieval, overcoming vocabulary gaps and aligning search results with how members naturally express their career ambitions.

Deploying LLMs at LinkedIn’s scale—serving millions of real-time queries per second—requires innovation that balances quality and efficiency. In this blog, we’ll share how we transformed our overarching search experience at LinkedIn, including the challenges and decisions that went into creating a scalable LLM-based stack and how the technology is powering a smarter, faster, and more personalized experience that helps every member find the most relevant opportunities and connections.

Semantic search’s high level infrastructure

When a member submits a query in the search bar, a query understanding module processes the input text, creates a query embedding, and performs embedding-based retrieval (EBR) on CUDA-enabled GPUs using exhaustive vector search ([paper on GPU CUDA-based Search](#), [paper on GPU PyTorch-based Search](#)) to assemble a broad set of candidate documents. The ranking stage then refines these candidates through a Cross-Encoder Small Language Model (SLM) deployed on SGLang, which combines the query, job, and member features to generate relevance and engagement scores for final ranking.

To maintain scalability and efficiency, the ranking pipeline integrates several optimization techniques ([paper on efficient LLM inference infrastructure](#), [context compression paper](#)): score caching, a ranking-depth controller to manage how many candidates progress to deeper ranking,

and traffic shaping to balance load during peak times—all designed to enhance latency and result quality. The features and concise job representations consumed by the SLM are produced through a hybrid inference pipeline: a large-scale offline workflow using Spark and Flyte, and a low-latency nearline system using Flink. These embeddings and summaries are stored in distributed storage and retrieved on demand with minimal latency.

In the final stage, the auction layer applies budget and pacing strategies to balance user relevance, engagement, and business metrics, ensuring a healthy equilibrium between recall and precision while maximizing member satisfaction. Figure 1 below provides an overview of the system architecture.

Product policy relevance measurement

Measuring relevance quality is essential to delivering a great search experience on LinkedIn. We define product policies that specify how to rate each query–document pair on a five-point scale and use LLM judges to apply these ratings at a massive scale, far beyond what manual evaluation can achieve. These judges are tightly aligned with product managers and engineers through iterative feedback to ensure high agreement. They not only grade tens of millions of query–document pairs daily for relevance measurement but also generate labeled data for training our retrieval and ranking systems, ensuring we optimize search quality according to product policy.

Defining product policy and golden product manager grades

A strong LLM judge begins with a clear product policy and high-quality product manager “golden” grades that demonstrate how that policy should be applied. Product managers act as a “Supreme Court,” regularly calibrating to resolve judgment differences and maintain a shared definition of what constitutes a good query–document match. These discussions refine the policy, making it clearer and less subjective. Once product managers reach high agreement (weighted Cohen’s Kappa ≥ 0.8), their labels are considered reliable ground truth.

To build a comprehensive golden dataset across diverse user queries, we first categorize queries by attributes (e.g., title–company, name–company, title–skill). Each category is then split into existing user queries and aspirational queries that represent strategic areas we want to excel in. We stratify-sample query–document pairs from each bucket to ensure broad coverage before sending them to product managers for grading.

Training the LLM judge

Our LLM judge must meet two requirements: high agreement with product managers and the ability to grade tens of millions of query–document pairs daily. To maximize agreement, we collaborate with product managers to prompt-engineer state-of-the-art LLMs, optimizing the weighted Cohen’s Kappa Score on golden data. The prompt encodes product-policy guidelines and few-shot examples to drive consistency. While these large models produce high-quality judgments, they cannot meet our throughput needs. To scale, we distill them into a smaller 8B-parameter evaluator LLM. Through supervised fine-tuning on a diverse dataset spanning all query categories and grades, we maintain only small drops in agreement—verified via the Kappa Score on the golden set—while achieving massive efficiency gains.

Continuously measuring quality of search system

Once we have our scalable LLM judge, we can finally build continuous relevance measurement of our system. On a regular basis, we build workflows that perform the following steps:

1. Stratify sample or synthesize a diverse set of queries based on the query categories defined earlier
2. Retrieve the documents returned from executing those queries
3. Decorate documents with additional information required for the correct evaluation/judgement
4. Grade the documents returned using our LLM judge
5. Calculate aggregate precision, recall, and NDCG metrics

This workflow serves three primary functions:

- Continuously monitoring the relevance of the overall system
- Evaluating experiments involving underlying ranking and retrieval subsystems
- Distilling student ranking and retrieval models by leveraging evaluation results

Search quality modeling

Search quality is a fundamental requirement for any search product, and achieving it depends on building the core components of a modern search engine. Below, we describe how we leveraged LLMs to enable query understanding, semantic embedding-based retrieval, and cross-encoder ranking.

Embedding-based retrieval

Retrieval is the stage of a search system that identifies a broad set of potentially relevant results from a large corpus. Because no search engine can score every document for every query in real time, we need an efficient retrieval layer to narrow the search space before ranking. Our retrieval sits on top of our GPU-enabled embedding-based retrieval (EBR) system. We built the EBR model by fine-tuning an open-source LLM embedding model to encode queries and jobs into dense vectors. We train on millions of real query-job pairs sampled from production logs, with relevance labels provided by an LLM-based judge. Each query includes its natural language text and query-understanding tags (e.g. workplace type or company), and each job is represented by structured metadata (title, company) plus its description. Importantly, this work also demonstrates a practical path for deploying LLM-based components in real production search systems. The semantic search can directly understand human language as queries, enabling much more intuitive search experiences. This has been a particularly inspiring aspect of the project: bringing modern LLM capabilities into a high-scale, real-time application that serves millions of users.

EBR relevance modeling

To ensure consistency between training and serving, every query is formatted using a lightweight prompt template:

```
Instruct: Given a job search query, retrieve relevant job postings
Query: {query}
{Optional Aspect eg. Company}: {company}
```

The model uses a dual-tower (bi-encoder) architecture: one encoder maps queries to embeddings and the other maps jobs, projecting them into a shared semantic space. Training is end-to-end: we fine-tune all model parameters using Hugging Face Accelerate (with PyTorch FSDP) across multiple GPUs.

We optimize a contrastive InfoNCE loss combined with a margin-based ranking loss. For example, given a query q , a positive job $d+$, and negatives $\{d-k\}$, we define:

where $\text{sim}(\cdot, \cdot)$ is the dot-product similarity of embeddings and τ is a temperature parameter. We also enhance training with hard positives and hard negatives mined from LLM-judged data. Hard positives are LLM-labeled relevant jobs that the current EBR model ranks low, and hard negatives are non-relevant jobs that the model ranks high. These examples reveal exactly where the model struggles. We then build targeted positive–negative pairs. A pairwise margin loss is applied to these curated cases to explicitly lift hard positives and suppress hard negatives, improving ranking where it matters most:

which encourages $\text{sim}(q, d_+)$ to exceed $\text{sim}(q, d_-)$ by a margin γ . The total loss is a weighted sum, e.g.:

combining the benefits of contrastive and pairwise ranking.

We use multiple evaluation pipelines. First, we leverage production logs for *counterfactual* evaluation: re-ranking the historical candidate list for each query and computing precision, recall and NDCG against true labels. Second, we run offline KNN simulations: we embed held-out queries and job corpus, retrieve nearest neighbors, and directly measure retrieval metrics on this test set. Finally, we integrate the new model into our online serving stack on a subset of traffic to collect end-to-end metrics. These offline metrics provide quick feedback, and the counterfactual log analysis helps estimate the model’s user impact without a full live experiment.

Productionization of retrieval

In production, we precompute and store all job embeddings in GPU-backed indexes. At runtime, the incoming query is encoded by the LLM with the prompt aligned with the LLM pre-training to produce its embedding. We then perform an exhaustive k-nearest-neighbor search over the job embedding index using dot-product similarity, returning the top-K jobs. This offline indexing and fast online query encoding makes retrieval extremely efficient, enabling low-latency serving of industry-scale semantic search products.

Query understanding

At the moment a member enters a query into the search bar—the entry point of semantic search—we apply a unified LLM-based understanding layer that interprets intent and converts free text into structured signals. For both AI Job Search and AI-powered People Search, this layer uses fine-tuned 1.5–4B parameter models that meet LinkedIn’s latency requirements while delivering high-precision structured outputs ([paper](#)). A single model handles intent classification, facet extraction, and profile-aware rewriting, replacing multiple brittle NER and heuristic components. The resulting attributes (e.g., title, company, school, location) feed directly into retrieval and ranking.

An intelligent routing layer works alongside this. A lightweight encoder classifies query types at high QPS and performs policy-based safety checks before sending the query down the appropriate path—LLM-powered semantic interpretation for ambiguous inputs or efficient keyword retrieval for precise name and entity lookups.

Together, these components provide a consistent, centrally governed semantic interface for People and Job Search, boosting relevance, simplifying the system, and enabling Semantic Search to scale across LinkedIn’s global traffic.

Small language model ranking

The ranking module of semantic search utilizes a Small Language Model (SLM) to estimate how relevant a user’s search query q is to each retrieved job_i . The SLM follows a decoder-only architecture. For example, for job search we represent the structured attributes of a job — including its title, company, location, employment type, and remote-work status. Meanwhile people search uses information from the member’s profile including their name, company information, position information, educational information and location. For each query–job pair (q, i) , we build a structured prompt defined as

Here, the *system prefix* and *suffix* contain chat-template tags and explicit instructions guiding the model to determine whether the given job matches the query. When this prompt is passed through the decoder, it produces logits corresponding to the next token. Let $logit_{yes}$ and $logit_{no}$ represent the logits for the tokens “yes” and “no,” respectively. Following prior studies, we compute:

which yields probabilities used to rank job items by their relevance to the user’s query.

SLM training pipeline for relevance quality

Training of the SLM follows a multi-stage process. First, we distill the 7B-parameter teacher model into a compact 0.6B model that can generate graded relevance labels along with rationales. Next, the teacher’s ordinal grades are converted into “soft labels” (p_{yes}, p_{no}) , representing probabilistic supervision.

We then perform supervised fine-tuning (SFT) to minimize the Kullback–Leibler (KL) divergence between the teacher’s soft targets and the SLM’s predicted probabilities—effectively converting the reasoning-oriented model into a binary relevance classifier:

We construct training labels by sampling real query–item pairs from user interaction logs and annotating them using a full-scale Large Language Model (LLM). Each pair is evaluated through a structured prompt of the following format:

[CRITERIA]: <matching guidelines>

[EXAMPLES]: <reasoning and output format>

[QUERY]: <query text>

[CANDIDATE]: <item text>

Analyze the query–candidate pair and assess how well they align. Provide

The LLM’s response includes both a graded relevance score and an accompanying rationale, ensuring that the generated labels are interpretable and consistent with the defined matching criteria.

The model is trained on logged pairs query–job pairs for up to five epochs using the prompt structure. Since job descriptions dominate the input and vary substantially in length (median $\approx$ 900 tokens; maximum $>$ 2300), we truncate them to ensure the total prompt length does not exceed 2048 tokens during both training and inference. For evaluation, we use a holdout dataset labeled by the teacher model. The SLM ranks job candidates based on $p_{yes} \cdot p_{no}$, and system performance is measured using Normalized Discounted Cumulative Gain at rank 10 (NDCG@10).

Training for multiple objectives

We extend the training paradigm to predict both relevance and engagement within the model, we train a smaller cross-encoder language model (the SLM) using multi-teacher, multi-task distillation. The teachers include:

- The product policy LLM for relevance scoring.
- Other large models that predict member actions—such as job views, applies, recruiter accepts, or (in people search) view profile, connecting, messaging, or following.

For training at scale, we use multiple 1.7B teacher models. To make this feasible, we first distilled our 7B product policy model into a smaller 1.7B version, which serves as a strong but efficient teacher alongside others. During the training process, these teachers run in real time on sampled production query–document pairs, producing soft probability scores that act as supervision targets. The student SLM is trained with KL divergence loss to align its output distribution with the ensemble of teachers. This setup lets us train on real-world data at scale:

teacher models provide rich, nuanced probability signals, and the distilled student captures much of their reasoning capacity while remaining lightweight enough to serve millions of queries per second in production.

We have multiple steps of distillation to SLM described in Figure 3, and we show the metrics after distillation in Table 1.

	NDCG@10	Apply AUC	Click AUC
Relevance Teacher 1.7B	0.9484	-	-
Engagement Teacher 1.7B	-	0.8049	0.6772
SLM 0.6B (distilled)	0.9239	0.8007	0.6704

To make LinkedIn’s AI-powered People Search more transparent, we introduced semantic, context-aware snippets that show *why* a result matches a member’s query. Snippets highlight the most relevant terms and maintain low latency through lazy loading, caching. The approach uses semantic similarity between the query embedding and precomputed phrase embeddings (unigrams/bigrams) from profile text, surfacing the highest-scoring phrases as natural, human-readable snippets. In the offline pipeline, we extract phrases from each profile section (e.g., summary, experience, education), encode them into embeddings in a [Venice key-value store](#). This index is refreshed periodically to capture profile updates, with evaluation workflows ensuring quality and readability. At search time, the snippetting midtier receives the query embedding and candidate profiles, fetches stored phrase embeddings, computes cosine similarity, selects top phrases, and expands them into readable snippets using simple heuristics. The final output is a ranked list of profiles paired with snippets and highlighting metadata.

To improve transparency in search results, we introduced a lightweight reasoning module that explains how LinkedIn interprets a member’s query. When a query is submitted, the LLM-based understanding model extracts facets, classifies intent, and—using predefined guidelines—produces a concise “thinking state” describing how the query is parsed, along with a brief summary of the types of profiles retrieved. For unsupported or negatively intended queries, the module instead provides a clear explanation of why results may be limited. For example, “berkeley community development specialist msa professional services” becomes “Searching for community development specialist at MSA Professional Services affiliated with Berkeley.” To keep latency low, reasoning outputs are cached in Couchbase and reused for repeated or semantically similar queries.

SLM ranking model inference efficiency

We employed multiple techniques to improve efficiency of the inference system at LinkedIn including:

- Model pruning, where we remove Fully Connected Layers and remove whole transformer layers.

Model pruning

To boost inference throughput, we apply model compression via structured pruning—a technique that removes redundant components to reduce model size and computation with minimal quality loss. Because our models run on GPU infrastructure, we focus on structured pruning, which removes entire neurons, attention heads, or transformer layers so the resulting model can run efficiently on standard GPU kernels. This yields real throughput and latency gains, unlike unstructured pruning, which drops individual weights but often provides no meaningful speedup without specialized hardware.

We prune hidden neurons in Multi-layer Perceptron (MLP) blocks and attention heads in self-attention modules, and we remove full transformer layers to study trade-offs between size, efficiency, and performance ([paper](#)). Pruning MLP neurons shrinks intermediate activations, while pruning layers reduces network depth—both producing lighter, faster models. After pruning, we fine-tune the model to recover any accuracy loss, ensuring efficiency improvements do not compromise quality.

Context pruning

Item descriptions are long (median ~900 tokens and up to 2,300) making them over 94% of the SLM prompt and causing ~10% of inputs to be truncated at the 2,048-token limit. Removing descriptions severely degrades relevance quality, confirming they carry essential semantic information. To handle this, we use a slightly larger 1.7B LLM to summarize descriptions offline, where item-specific inference can be precomputed and refreshed via streaming updates. Because job descriptions include verbose, often irrelevant details, we train the summarizer with a semantics-preserving loss and a length-aware reward ([paper](#)). We fine-tune the model using RL to produce concise summaries, balancing (1) reduced input length and (2) preserved model quality. The reward combines:

- a semantic consistency term, measured via KL divergence between the SLM’s output distributions on summarized vs. raw text, and

- a length penalty that discourages overly long summaries.

The weighting factor w controls the trade-off between brevity and fidelity, enabling summaries that retain meaning while reducing inference cost.

Embedding compression

Since the computational cost of LLM inference increases quadratically with input length, reducing the number of tokens can greatly decrease overall inference expense. On top of model pruning and summarization, we invented a text–embedding hybrid interaction architecture that condenses each item’s text into a single-token embedding generated by an encoder LLM ([paper](#)). These embeddings are then merged with other textual signals and passed to a ranker LLM for relevance estimation. Because the item embeddings are precomputed and stored in a nearline cache, the volume of text processed during online inference is significantly reduced, resulting in notable improvements in efficiency.

We jointly train the hierarchy of two language models, where one model is producing embedding of the item description and the other model does the ranking of the candidates. On the figure below we show we use two 0.6B models trained jointly. As a result we could replace most of the job description with just a single embedding. We may keep important raw fields with a limited number of text tokens such as title of the job, company name, location or member name in the input.

Overall modeling quality and throughput

In the table below we show how quality and inference throughput changed as we improved the modeling technology. As part of the modeling improvement we introduced multitask learning with over 6 tasks of member actions to the model output predictions including relevance, and, for example, were able to improve Click AUC of the model from 0.61 ([baseline](#)) to 0.67 for Job Ranking.

Setup	NDCG@10 Throughput (ITEMS/SEC/GPU)	
SLM with raw-text	0.9432	290
Pruned SLM and SUMMARIZED TEXT	0.9218	2200
SLM with EMBEDDING COMPRESSION	0.9239	22000
Embedding based retrieval (EBR) baseline	0.838	>1.6B, exhaustive search on GPU

Looking ahead

We built a modern semantic search system by first establishing an LLM-based quality evaluation framework grounded in product policy. On top of this foundation, we introduced LLM-powered query understanding, semantic retrieval, and ranking models—driving double-digit improvements in search quality and member engagement. Just as importantly, we achieved this while keeping inference costs comparable to traditional RecSys models previously running in production. As we continue refining Semantic Search, our focus remains on empowering every member to discover the right opportunity, connection, or insight—at the right time.

Acknowledgements

To the more than 100 team members who've contributed to AI-powered job search and people search across infrastructure, AI modeling, user experience, and data science: thank you for your creativity, grit, and teamwork. This milestone belongs to all of us.